

assignment0

January 28, 2026

1 CS260R Reinforcement Learning Assignment 0: Jupyter Notebook usage and assignment submission workflow

CS260R: Reinforcement Learning. Department of Computer Science at University of California, Los Angeles. Course Instructor: Professor Bolei ZHOU. Assignment author: Zhenghao PENG.

You are asked to finish four tasks:

1. Fill in your name and University ID in the next cell.
2. Install pytorch and finish the [Kindergarten Pytorch](#) section.
3. Run all cells and export this notebook **as a PDF file**.
4. Compress this folder **assignment0 as a ZIP file** and submit **the PDF file and the ZIP file separately as two files** in BruinLearn.

```
[1]: # TODO: Fill your name and UID here
my_name = "Mikey"
my_student_id = "auditing"
```

```
[2]: # Run this cell without modification

text = "Oh, I finished this assignment! I am {} ({}).".format(my_name,
    ↪my_student_id)
print(text)
with open("{}_txt".format(text), "w") as f:
    f.write(text)
```

Oh, I finished this assignment! I am Mikey (auditing)

1.1 Kindergarten Pytorch

1. Please install pytorch in your virtual environment following the instruction: <https://pytorch.org/get-started/locally/>.

```
pip install torch torchvision
```

2. If you are not familiar with Pytorch, please go through [the tutorial in official website](#) until you can understand the [quick start tutorial](#).
3. The following code is copied from the [quick start tutorial](#), please solve all TODOs and print the result in the cells before generating the PDF file.

[3]: *# You can also run this cell in notebook:*

```
!pip install torch torchvision
```

```
Requirement already satisfied: torch in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (2.10.0)
Requirement already satisfied: torchvision in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (0.25.0)
Requirement already satisfied: filelock in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (from
torch) (3.20.3)
Requirement already satisfied: typing-extensions>=4.10.0 in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (from
torch) (4.15.0)
Requirement already satisfied: sympy>=1.13.3 in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (from
torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (from
torch) (3.6.1)
Requirement already satisfied: jinja2 in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (from
torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (from
torch) (2026.1.0)
Requirement already satisfied: numpy in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (from
torchvision) (2.4.1)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (from
torchvision) (12.1.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (from
sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/Users/sttawm/opt/miniconda3/envs/cs260r/lib/python3.11/site-packages (from
jinja2->torch) (3.0.3)
```

1.1.1 Prepare data

```
[4]: import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor

# Download training data from open datasets.
```

```

training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor(),
)

# Download test data from open datasets.
test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor(),
)

batch_size = 64

# Create data loaders.
train_dataloader = DataLoader(training_data, batch_size=batch_size)
test_dataloader = DataLoader(test_data, batch_size=batch_size)

```

1.1.2 Define model

```

[5]: # Get cpu, gpu or mps device for training.
device = (
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)
print(f"Using {device} device")

# Define model
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()

        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28 * 28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10)
        )

```

```

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

model = NeuralNetwork().to(device)
print(model)

```

Using mps device

```

NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)

```

1.1.3 Define training and test pipelines

```

[6]: loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)

def train(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        X, y = X.to(device), y.to(device)

        # Compute prediction error
        pred = model(X)
        loss = loss_fn(pred, y)

        # Backpropagation
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        if batch % 100 == 0:
            loss, current = loss.item(), (batch + 1) * len(X)
            print(f"loss: {loss:>7f} [{current:>5d}/{size:>5d}]")

def test(dataloader, model, loss_fn):
    size = len(dataloader.dataset)
    num_batches = len(dataloader)

```

```

model.eval()
test_loss, correct = 0, 0
with torch.no_grad():
    for X, y in dataloader:
        X, y = X.to(device), y.to(device)
        pred = model(X)
        test_loss += loss_fn(pred, y).item()

        # TODO: Uncomment next line and understand what it means
        correct += (pred.argmax(1) == y).type(torch.float).sum().item()

test_loss /= num_batches
correct /= size
print(f"Test Error: \n Accuracy: {(100*correct):>0.1f}%, Avg loss:␣
↪{test_loss:>8f} \n")

```

1.1.4 Run the training and test pipelines

```

[7]: epochs = 5
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train(train_dataloader, model, loss_fn, optimizer)
    test(test_dataloader, model, loss_fn)
print("Done!")

```

Epoch 1

```

-----
loss: 2.309081 [ 64/60000]
loss: 2.290136 [ 6464/60000]
loss: 2.275687 [12864/60000]
loss: 2.275269 [19264/60000]
loss: 2.251581 [25664/60000]
loss: 2.230027 [32064/60000]
loss: 2.228283 [38464/60000]
loss: 2.195453 [44864/60000]
loss: 2.190290 [51264/60000]
loss: 2.169561 [57664/60000]
Test Error:
Accuracy: 47.6%, Avg loss: 2.156332

```

Epoch 2

```

-----
loss: 2.161809 [ 64/60000]
loss: 2.153620 [ 6464/60000]
loss: 2.092623 [12864/60000]
loss: 2.122403 [19264/60000]
loss: 2.064668 [25664/60000]
loss: 2.002341 [32064/60000]

```

loss: 2.034405 [38464/60000]
loss: 1.945848 [44864/60000]
loss: 1.955410 [51264/60000]
loss: 1.894488 [57664/60000]
Test Error:
Accuracy: 55.6%, Avg loss: 1.880333

Epoch 3

loss: 1.912729 [64/60000]
loss: 1.886844 [6464/60000]
loss: 1.756899 [12864/60000]
loss: 1.817196 [19264/60000]
loss: 1.704948 [25664/60000]
loss: 1.647946 [32064/60000]
loss: 1.686504 [38464/60000]
loss: 1.570674 [44864/60000]
loss: 1.601606 [51264/60000]
loss: 1.514025 [57664/60000]
Test Error:
Accuracy: 58.4%, Avg loss: 1.517589

Epoch 4

loss: 1.585372 [64/60000]
loss: 1.557674 [6464/60000]
loss: 1.394692 [12864/60000]
loss: 1.483092 [19264/60000]
loss: 1.369133 [25664/60000]
loss: 1.352894 [32064/60000]
loss: 1.381073 [38464/60000]
loss: 1.289986 [44864/60000]
loss: 1.326624 [51264/60000]
loss: 1.246884 [57664/60000]
Test Error:
Accuracy: 62.2%, Avg loss: 1.261847

Epoch 5

loss: 1.338732 [64/60000]
loss: 1.327947 [6464/60000]
loss: 1.154049 [12864/60000]
loss: 1.269137 [19264/60000]
loss: 1.151757 [25664/60000]
loss: 1.164373 [32064/60000]
loss: 1.192539 [38464/60000]
loss: 1.115592 [44864/60000]
loss: 1.155836 [51264/60000]

```
loss: 1.090588 [57664/60000]
Test Error:
  Accuracy: 64.2%, Avg loss: 1.102099
```

Done!

1.1.5 Save model

```
[8]: torch.save(model.state_dict(), "model.pth")
      print("Saved PyTorch Model State to model.pth")
```

Saved PyTorch Model State to model.pth

1.1.6 Load model and run the inference

```
[9]: model = NeuralNetwork().to(device)
      model.load_state_dict(torch.load("model.pth"))
```

[9]: <All keys matched successfully>

```
[10]: classes = [
        "T-shirt/top",
        "Trouser",
        "Pullover",
        "Dress",
        "Coat",
        "Sandal",
        "Shirt",
        "Sneaker",
        "Bag",
        "Ankle boot",
      ]

      model.eval()
      x, y = test_data[0][0], test_data[0][1]
      with torch.no_grad():
          x = x.to(device)
          pred = model(x)
          predicted, actual = classes[pred[0].argmax(0)], classes[y]
          print(f'Predicted: "{predicted}", Actual: "{actual}"')
```

Predicted: "Ankle boot", Actual: "Ankle boot"

```
[ ]:
```